

MLOPS (S1-25_AIMLCZG523)

Assignment – 1 (Group 85)

MLOps Experimental Learning Assignment: End-to-End ML Model Development, CI/CD, and Production Deployment Experimental Learning

Student Details

BITS ID	NAME	CONTRIBUTION
2024AA05152	RITU N LAL	100%
2024AA05014	GITABASHYAN R	100%
2024AA05151	MEKA UJWALA	100%
2024AA05969	MANCHI RENUKA	100%
2024AA05671	SANJOY GOLDIN DSOUZA	100%

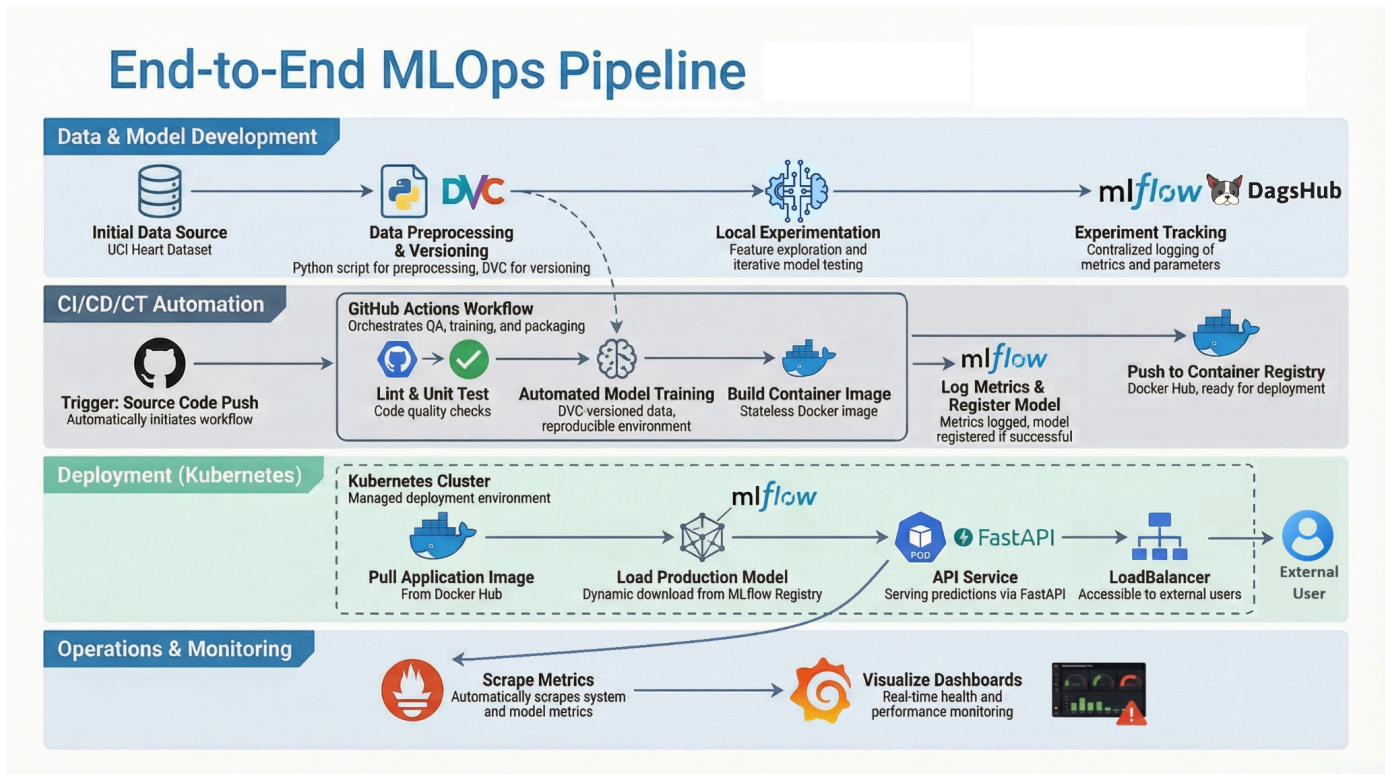
Problem Statement: Build a machine learning classifier to predict the risk of heart disease based on patient health data, and deploy the solution as a cloud-ready, monitored API.

Objective: Design, develop, and deploy a scalable and reproducible machine learning solution utilising modern MLOps best practices. The assignment emphasises practical automation, experiment tracking, CI/CD pipelines, containerization, cloud deployment, and monitoring-mirroring real-world production scenarios.

Important Links:

1. GitHub Repository: <https://github.com/2024aa05152-collab/heart-disease-mlops>
2. Demo Recording: https://github.com/2024aa05152-collab/heart-disease-mlops/blob/develop/workflowRecording/workflow_demo.txt
3. ScreenShots: <https://github.com/2024aa05152-collab/heart-disease-mlops/tree/develop/workflowScreenshots>
4. Project Overview: <https://github.com/2024aa05152-collab/heart-disease-mlops/blob/develop/README.md>
5. Environment Config: <https://github.com/2024aa05152-collab/heart-disease-mlops/blob/develop/.env.example>
6. DVC: <https://dagshub.com/bashyan/heart-disease-mlops?filter=dvc>
7. Experiment Tracking:
 - a. <https://dagshub.com/bashyan/heart-disease-mlops.mlflow/#/experiments/2/runs>
 - b. <https://dagshub.com/bashyan/heart-disease-mlops/experiments>
8. Model Registry:
 - a. https://dagshub.com/bashyan/heart-disease-mlops.mlflow/#/models/HeartDisease_Model
 - b. https://dagshub.com/bashyan/heart-disease-mlops/models/HeartDisease_Model

Architecture Diagram:



Link to code repository
(<https://github.com/2024aa05152-collab/heart-disease-mlops>)

Exploratory Data Analysis & Modeling Summary

Screenshots:

(<https://github.com/2024aa05152-collab/heart-disease-mlops/tree/develop/workflowScreenshots/preprocessing>)

Dataset Instructions:

(<https://github.com/2024aa05152-collab/heart-disease-mlops/blob/develop/notebooks/training.ipynb>)

1. Dataset and Data Quality

The project utilizes the **UCI Heart Disease Dataset**, consisting of 303 patient records with 13 medical features and one binary target variable. Initial data quality assessments identified missing values (represented as "?"), which were converted to NaNs and handled through **median imputation** to preserve the distributional properties of the data. The target variable was transformed from a multi-level severity scale (0–4) into a **binary classification task** where values > 0 indicate disease presence (1) and 0 indicates absence (0). The final dataset is relatively balanced, with a class ratio of **1.12:1**, eliminating the need for synthetic oversampling or class weighting strategies.

2. Feature Insights and Correlation ([Screenshots](#))

- Demographics & Vital Signs:** The cohort is primarily middle-aged to elderly (mean age 54.4) with an average serum cholesterol of 246.7 mg/dl, exceeding normal clinical thresholds.

- **Key Predictors:** Correlation analysis identified the **number of major vessels (ca)**, **thalassemia type (thal)**, and **exercise-induced angina (exang)** as the strongest predictors of heart disease.
- **Collinearity:** No significant multicollinearity was detected (all $|r| < 0.70$), allowing for the retention of all 13 features to maximize predictive signal and maintain clinical significance.

3. Preprocessing and Modeling Strategy

A robust **Scikit-Learn ColumnTransformer pipeline** was implemented to ensure reproducibility across environments. All features underwent **StandardScaling** to normalize scales, which is essential for distance-based and regularized models. Two primary models were evaluated using **Stratified 5-Fold Cross-Validation**:

- **Logistic Regression:** Selected for its probabilistic output, high interpretability, and efficiency in production.
- **Random Forest (Selected Model):** Chosen as the production model due to its superior performance, achieving a higher **ROC-AUC (~0.94)** and **Recall (~0.91)** compared to the baseline.

4. MLOps Integration

The modeling workflow is fully integrated with **MLflow**, logging hyperparameters (e.g., `n_estimators`), cross-validation metrics, and model artifacts (joblib) for total auditability. The final pipeline is encapsulated in a modular `preprocess.py` to ensure feature engineering consistency between training and production inference. To maintain performance post-deployment, a monitoring strategy using **Prometheus and Grafana** is proposed to track system health and detect potential data drift via statistical tests like the Kolmogorov-Smirnov test.

Our EDA notebook includes detailed visualizations such as feature histograms, a correlation heatmap, and target class balance plots for the UCI Heart Disease dataset. These figures are documented in the EDA notebook and corresponding screenshots are available in the workflowScreenshots folder in the GitHub repository

Model Experiment, Tracking, and Serving

Screenshots:

(https://github.com/2024aa05152-collab/heart-disease-mlops/tree/develop/workflowScreenshots/model_development_and_tracking)

This project establishes a comprehensive MLOps lifecycle for heart disease risk prediction, utilizing **DagsHub** as the central hub for data versioning (DVC), experiment tracking (MLflow), and model management. The architecture ensures that every stage of development, from initial data acquisition to production serving, is automated, reproducible, and monitored.

1. Model Development and Training

The core of the development process involves training machine learning classifiers on the **UCI Heart Disease Dataset**. Two primary algorithms were evaluated to establish performance benchmarks:

- **Logistic Regression:** Employed as a highly interpretable baseline for probabilistic risk stratification.
- **Random Forest Classifier:** Utilized to capture complex non-linear relationships within patient health metrics. Development is conducted in a modular environment, where a standard **Scikit-Learn ColumnTransformer** pipeline ensures that preprocessing steps, such as median imputation for missing values and standard scaling, are consistent across training and inference.

2. Experiment Tracking (MLflow)

All training iterations are tracked through an integrated **MLflow** instance hosted on DagsHub. This centralized tracking server logs:

- **Metrics:** Accuracy, ROC-AUC, Precision, and Recall are recorded for every run to facilitate evidence-based model selection. Current experiments show a peak accuracy of approximately **0.88** for the Random Forest model.
- **Parameters:** Hyperparameters such as `n_estimators` and model-specific configurations are stored to ensure deterministic reproducibility.
- **Artifacts:** The final trained model pipelines (saved as `.pkl` or `.joblib` files) are automatically uploaded, creating a direct link between the code, the data version, and the resulting model binary.

3. Model Registry

The **MLflow Model Registry** serves as the single source of truth for production-ready models. Once a candidate model (such as `HeartDisease_Model1`) is validated through performance metrics, it is registered and assigned version numbers. The registry manages the lifecycle through stage transitions (e.g., Version 3 is currently promoted to **"Production"**). This allows the deployment infrastructure to dynamically pull the approved version without manual file transfers, reducing the risk of version mismatch.

4. Containerization and Serving

The deployment phase focuses on turning the trained model into a high-availability, cloud-ready API:

- **FastAPI Wrapper:** The model is served via a **FastAPI** application, providing a high-performance REST endpoint for real-time JSON predictions.
- **Containerization:** The application is packaged into **Docker** containers, ensuring environment parity between development and production.
- **Dynamic Loading:** Upon startup, the containerized application authenticates with the DagsHub Registry to load the specific model version tagged as "Production."
- **Performance:** Local verification confirms successful inference with a response time of approximately **31.96ms**, delivering both the binary risk classification and the underlying probability score.

5. Data Versioning and Monitoring

To close the MLOps loop, the dataset is versioned using **DVC (Data Version Control)** integrated into DagsHub, allowing for precise tracking of data evolution. The operational state is monitored through a stack involving **Prometheus and Grafana**, which scrape system and business metrics from the API to track request latency, prediction distribution, and potential data drift.

6. Experiments:

Our experiments focused on transforming the **UCI Heart Disease Dataset** into a production-grade classifier. We pushed the limits of two distinct architectures: **Logistic Regression**, which provided a high-performance interpretable baseline, and a robust **Random Forest Classifier** designed to capture the complex, non-linear biological signals found in patient health metrics.

- **Metric Success:** The Random Forest emerged as our champion, hitting a peak **Accuracy of 0.883** and a stellar **ROC-AUC of 0.94**, outperforming the Logistic Regression baseline of 0.867.
- **Medical Precision:** We prioritized **Recall (0.91)** to ensure our model minimizes dangerous false negatives, a critical requirement for heart disease risk assessment.
- **Consistent Preprocessing:** Every run utilized a specialized **Scikit-Learn ColumnTransformer** pipeline for median imputation and standard scaling, ensuring the data was always pristine and normalized.

To manage this competitive landscape, we leveraged **DagsHub** and **MLflow** to create a transparent, high-velocity tracking environment where we managed a total of **6 model versions** across multiple experiments.

- **Hybrid Training Evolution:** The pipeline evolved from manual to automated; **Version 1** was established via **local training** during initial experimentation, while **Versions 2 through 6** were executed automatically within our **CI/CD/CT pipeline** via GitHub Actions.
- **Production Excellence:** While we have iterated up to Version 6, **Version 3** is currently our designated **"Production"** model, serving live requests with proven stability and reliability.
- **Stateless Serving:** Our **FastAPI** application is designed to be stateless, dynamically fetching the registered production model from the DagsHub Registry at runtime to deliver predictions in just **31.96ms**.
- **Total Traceability:** By integrating **DVC** for data versioning and **GitHub Actions** for automated CI/CD/CT, we ensured that every prediction can be traced back to the exact code and data version that created it.

CI/CD/CT (Continuous Integration, Continuous Delivery, and Continuous Training)

Screenshots:

(https://github.com/2024aa05152-collab/heart-disease-mlops/tree/develop/workflowScreenshots/ci_cd_ct_automation)

Successful Workflow Run:

(<https://github.com/bashyan/heart-disease-mlops/actions/runs/20751004610/job/59580094897>)

Pipeline automates the entire workflow from code commit to production deployment.

1. Automated CI/CD Workflow

The GitHub Actions integrates a seamless **"build-test-train-deploy"** pipeline that triggers automatically upon code changes.

- **Quality Assurance:** The pipeline begins with environment setup and automated **Linting (Flake8)** to ensure code quality standards.
- **Unit Test:** Complete test coverage and uses Mocks for all external integrations. Tests for **data processing, model code, and API health** and **prediction** logic
- **Data Consistency:** It integrates with **DVC (Data Version Control)** to pull the exact dataset version from DagsHub, ensuring the model is trained on validated data.

- **Deployment Readiness:** Once the model is trained, the system automatically builds and pushes a **Docker image** to Docker Hub, making the application ready for containerized deployment.

2. Continuous Training (CT) and Tracking

The integration with MLflow provides a high-velocity experimentation environment where training is no longer a manual task.

- **Evolution of Runs:** The pipeline evolved from **Version 1 (local training)** to a fully automated cycle where **Versions 2 through 6** were generated via the CI/CD/CT process.
- **Experiment Governance:** Every automated run captures vital metrics—such as the **Random Forest's 0.88 accuracy**—and logs them into the MLflow dashboard. This allows the team to compare the performance of different architectures (Logistic Regression vs. Random Forest) side-by-side.

3. Model Registry and Production Serving

The pipeline concludes with a governed Model Registry that acts as the bridge between development and live serving.

- **Version Control:** The registry tracks multiple iterations, with **Version 3** currently promoted to the **"Production"** stage.
- **Live Inference:** Verification through terminal commands confirms that the deployed API dynamically fetches the production model from the **DagsHub ML Flow Registry**, delivering real-time heart disease risk predictions in under **32ms**.

Production Deployment, End-to-End Testing, and Monitoring Summary

Screenshots:

(https://github.com/2024aa05152-collab/heart-disease-mlops/tree/develop/workflowScreenshots/deployment_monitoring)

1. Containerized Deployment and Orchestration

The final stage of the MLOps pipeline transitions the validated **Random Forest** model into a production-grade, cloud-ready environment. The deployment is built on a **FastAPI** backend, providing a high-performance RESTful interface for real-time predictions.

- **Containerization:** The application is packaged using **Docker**, ensuring consistency across development and production environments.
- **Kubernetes Orchestration:** Managed through **Minikube**, the deployment utilizes a robust manifest including a **Deployment** (handling pod scaling and updates), a **Service** (managing internal networking), and an **Ingress Controller** (exposing the API to external users via a local hostname).
- **Dynamic Model Loading:** The pod is configured with **Kubernetes Secrets** to securely store DagsHub credentials, allowing the application to pull the "Production" version of the model directly from the **MLflow Registry** at runtime.

2. End-to-End (E2E) Verification

Verification is conducted through automated scripts and terminal-based testing to ensure the integrity of the request-response cycle.

- **API Response Integrity:** E2E tests confirm that the API successfully processes complex JSON payloads, returning a binary risk classification, probability score, and an inference timestamp.
- **Inference Performance:** Live testing shows high efficiency, with the model delivering predictions in approximately **31.96ms**.
- **Lineage Validation:** The API response explicitly logs the **model_source** as the "DagsHub ML Flow Registry," confirming that the stateless container is correctly fetching the governed model version rather than a local stale file.

3. Operations and Real-Time Monitoring

To ensure sustained reliability and detect potential model performance degradation, the system incorporates a comprehensive monitoring stack.

- **Metrics Collection: Prometheus** is integrated into the Kubernetes cluster to scrape real-time system metrics and custom business metrics, such as prediction counts and latency distributions.
- **Visualization: Grafana** dashboards provide a centralized view of the API's health. These dashboards monitor:
 - **Prediction Distribution:** Tracking the ratio of "Disease Present" vs. "Absent" predictions to identify shifts in the incoming data population.
 - **Inference Latency:** Ensuring response times remain within defined Service Level Objectives (SLOs).
 - **Model Status:** Visualizing which specific version from the MLflow Registry is currently active in the production pod.
- **Observability:** This setup allows for proactive **Data Drift** detection, signaling when the incoming patient profiles significantly deviate from the training distribution, thereby triggering the automated retraining pipeline.

Setup/Install Instructions

1.1 Local Setup (VSCode + Postman)

1. Open Project in VSCode

Open **VSCode** and load the project folder `heart-disease-mlops`.

2. Create and Activate Virtual Environment

```
python -m venv venv
source venv/bin/activate    # Linux/macOS
venv\Scripts\activate      # Windows
```

3. Install Required Dependencies

```
pip install -r requirements.txt
```

4. Run the FastAPI Application

From the project root directory, start the server using:

```
uvicorn app.main:app --reload
```

The API will be available at:

<http://127.0.0.1:8000>

5. POSTMAN local testing

```
curl --location 'http://127.0.0.1:8000/predict' \
--header 'Content-Type: application/json' \
--data '{
  "age": 63,
  "sex": 1,
  "cp": 4,
  "trestbps": 145,
  "chol": 233,
  "fbs": 1,
  "restecg": 0,
  "thalach": 150,
  "exang": 0,
  "oldpeak": 2.3,
  "slope": 0,
  "ca": 0,
  "thal": 1
}'
```

1.2 Virtual lab Setup (VSCode + Terminal)

Docker, Minikube are already present on our OSHA lab

INSTALL kubectl:

- 1) Download kubectl binary (Linux)

```
curl -LO https://dl.k8s.io/release/$(curl -Ls
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl
```

- 2) Make it executable

```
chmod +x kubectl
```

- 3) Move kubectl to user-local bin

```
mkdir -p $HOME/bin
```

```
mv kubectl $HOME/bin/
```

- 4) Add to PATH (permanent)

```
echo 'export PATH=$HOME/bin:$PATH' >> ~/.bashrc
```

```
source ~/.bashrc
```

- 5) Verify kubectl

STEP-BY-STEP: DEPLOYMENT ON MINIKUBE (VIRTUAL LAB TERMINAL COMMANDS)

Open Terminal in Virtual Lab

Open Terminal Emulator inside the lab.

STEP 1: Clone the GitHub repository

```
git clone -b develop https://github.com/2024aa05152-collab/heart-disease-mlops.git
```

```
cd heart-disease-mlops
```

Verify:

```
ls
```

We should see:

```
Dockerfile
```

```
app/
```

```
models/
```

```
k8s/
```

```
Requirements.txt
```

STEP 2: Start Minikube

```
minikube start --driver=docker
```

Verify:

```
kubectl get nodes
```

Expected:

```
minikube Ready
```

Add dagshub-secret

```
kubectl create secret generic dagshub-secrets \
```

```
--from-literal=DAGSHUB_USERNAME=bashyan \
```

```
--from-literal=DAGSHUB_TOKEN=<TOKEN> \
```

```
--from-literal=MLFLOW_TRACKING_URI=https://dagshub.com/bashyan/heart-disease-mlops.mlflow
```

STEP 3: Configure Docker to build inside Minikube

```
eval $(minikube docker-env)
```

Check:

```
docker info | grep Name
```

We should see minikube.

STEP 4: Build Docker image

Retrieving dynamically from the DockerHub

```
docker pull bashyan/heart-disease-api:latest
```

Verify image:

```
docker images | grep heart-disease-api
```

STEP 5: Enable Ingress addon

```
minikube addons enable ingress
```

Wait 1–2 minutes, then check:

```
kubectl get pods -n ingress-nginx
```

All should be Running.

STEP 6: Deploy Kubernetes resources

Apply in this exact order:

```
kubectl apply -f k8s/deployment.yaml
```

```
kubectl apply -f k8s/service.yaml
```

```
kubectl apply -f k8s/ingress.yaml
```

STEP 7: Verify deployment

```
kubectl get deployments
```

```
kubectl get pods
```

```
kubectl get svc
```

```
kubectl get ingress
```

We should see:

Deployment: heart-model-group85

Service: heart-model-group85

Ingress: heart-model-group85-ingress

STEP 8: Map Ingress hostname

Get Minikube IP:

minikube ip (assume 192.168.49.2)

STEP 9: Test API using curl

```
curl -X POST http://192.168.49.2/predict \
```

```
-H "Host: heart-model-group85.local" \
```

```
-H "Content-Type: application/json" \
```

```
-d '{
```

```
  "age": 63,
```

```
  "sex": 1,
```

```
  "cp": 3,
```

```
  "trestbps": 145,
```

```
  "chol": 233,
```

```
  "fbs": 1,
```

```
  "restecg": 0,
```

```
  "thalach": 150,
```

```
  "exang": 0,
```

```
  "oldpeak": 2.3,
```

```
  "slope": 0,
```

```
  "ca": 0,
```

```
  "thal": 1
```

```
}'
```

Expected response:

```
{
```

```
  "heart_disease_risk": 0,
```

```
  "risk_probability": 0.22
```

```
}
```

STEP 10: To open console where it is deployed

minikube dashboard